Hodgkin-Huxley and Firing Neurons

Joshua S. Weitz[1, 2, *]

[1] *School of Biology, Georgia Institute of Technology, Atlanta, GA, USA*
[2] *School of Physics, Georgia Institute of Technology, Atlanta, GA, USA*

(Dated: October 2, 2020)

I. COMPUTATIONAL NEURON-SCIENCE

The field of computational neuroscience stretches broadly, from neurons to networks and to models of cognition. Yet, at the core of neuroscience are units of cellular processing: neurons. These neurons are the 'nodes' of neural networks. Electrical signals processed at neurons propagate outwards along axons and then form the input signals via transmission through dendrites into other neurons. Critically, neurons are not linear transmitters. That is, they do not merely pass on inputs into outputs. Rather, they *process* signals in a nonlinear fashion. This nonlinearity includes the functional ability to filter out signals and to amplify others. As a consequence, many neurons acting together can respond to signals, store information, recall information, enable short- and long-term memory, and even enable the ability to forget. The origins of nonlinear information processing lies in the fact that neurons are, themselves, dynamical systems. A dynamical system can have the same governing parameters yet be in a different state depending on past history. This also means that the answer to understanding why a particular neuron responds to the same input in a different way might not lie in intrinsic differences in structure, but rather in differences in connectivity, feedback, and its state. That insight underlies the goal of this laboratory: to implement a computational representation of the neuron as a nonlinear dynamical system following upon the seminal work of Hodgkin and Huxley.

The Hodgkin-Huxley equations denote changes in four components of a neuron, the trans-membrane voltage, V, and three gating variables, n, m, and h, associated with the probability that three ionic channels are in the 'open' status: potassium, sodium and a leak (predominantly 'chloride'):

$$\overbrace{C_M \frac{\mathrm{dV}}{\mathrm{dt}}}^{\text{capacitive current}} = \overbrace{I}^{\text{applied current}} - \overbrace{\bar{g}_K n^4 (V - E_K)}^{\text{potassium current}} - \overbrace{\bar{g}_{Na} m^3 h (V - E_{Na})}^{\text{sodium current}} - \overbrace{\bar{g}_l (V - E_l)}^{\text{leak current}} \tag{1}$$

$$\overbrace{\frac{\mathrm{dn}}{\mathrm{dt}}}^{\text{potassium channels}} = \overbrace{\overbrace{\alpha_n(1-n)}^{\text{on}} - \overbrace{\beta_n n}^{\text{off}}} \tag{2}$$

$$\overbrace{\frac{\mathrm{dm}}{\mathrm{dt}}}^{\text{sodium channels}} = \overbrace{\overbrace{\alpha_m(1-m)}^{\text{on}} - \overbrace{\beta_m m}^{\text{off}}} \tag{3}$$

$$\overbrace{\frac{\mathrm{dh}}{\mathrm{dt}}}^{\text{leak channels}} = \overbrace{\overbrace{\alpha_h(1-h)}^{\text{on}} - \overbrace{\beta_h h}^{\text{off}}} \tag{4}$$

The details and derivation of these equations are found in the textbook (see Figure 1 for a schematic).

Briefly, the phrase above each term in the equations helps to explain the context. The dynamics arise from three physical effects: (i) Ohm's law, i.e., that a voltage is equal to the current multiplied by the resistance (or the inverse of capacitance); (ii) Kirchhoff's rule, i.e., that currents are conserved; (iii) the opening and closing of ionic channels

*Electronic address: jsweitz@gatech.edu; URL: http://ecotheory.biology.gatech.edu

Typeset by REVTEX

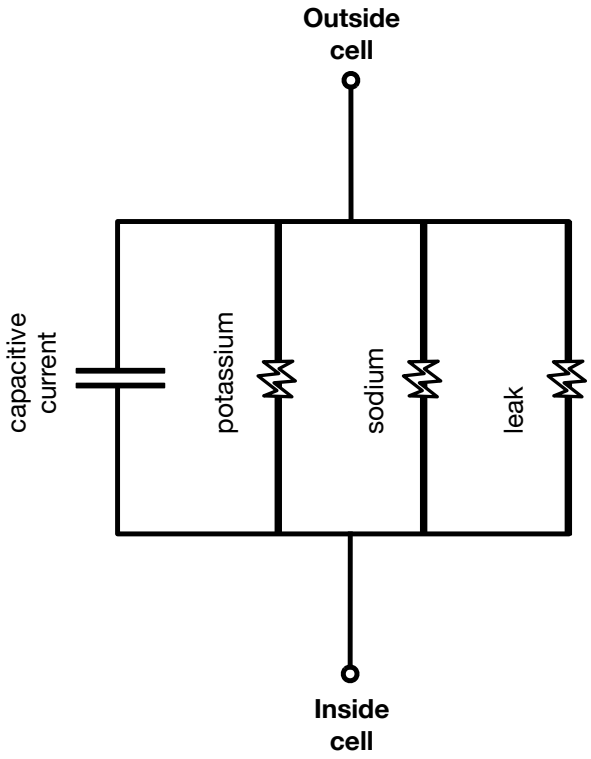


FIG. 1: Representation of the transfer of ions across a cell membrane via ion channels and via the a capacitive current, reflecting the ability of the membrane to hold a charge difference.

are voltage-dependent, i.e.,:

$$\alpha_n(V) = 0.01\frac{10-V}{e^{\frac{10-V}{10}}-1} \quad (5)$$

$$\beta_n(v) = 0.125e^{-V/80} \quad (6)$$

$$\alpha_m(V) = 0.1\frac{25-V}{e^{\frac{25-V}{10}}-1} \quad (7)$$

$$\beta(m)(V) = 4e^{-V/18} \quad (8)$$

$$\alpha_h(V) = 0.07e^{-V/20} \quad (9)$$

$$\beta_h(V) = \frac{1}{e^{\frac{30-V}{10}}+1}. \quad (10)$$

Throughout these equations, voltage is measured in units of millivolts (mV).

These equations reflect a tour de force of experiments and modeling – but they are also opaque, at least at first encounter. Indeed what is possible for a neuron to do, in a dynamic sense, may not appear obvious by simply looking (or admiring) these equations. But that's okay. These equations are not meant to be admired. Instead, they represent a biophysical representation of how a neuron responds to stimuli, and they are meant to be used, explored, and compared to experiments. Hence, the goal of this lab is to simulate neuronal dynamics and to identify some of the key dynamical features of this excitatory system. In doing so, the lab will provide a guide to modulate the Hodgkin-Huxley model, see how it works in practice, and gain intuition for concepts including: (i) sub-critical inhibition; (ii) super-critical firing; (iii) and refractory behavior. Unlike prior laboratories, simply writing in the code takes time, and so it is recommended that some component of this lab are pre-loaded prior to starting. Some of these pre-loaded codes have missing components; that is intentional to give you an opportunity to fill in the gaps. Even if you have the code in-hand, please be sure to read through each line. There is subtlety here, and editing and modifying the code will help build your understanding of computational neuron-science in a way that merely hearing about what neurons can do never will.

II. HODGKIN-HUXLEY MODEL

In this section, you will "experiment" with the HH model. In doing so, you will have a chance to build your intuition for the mechanisms by which a neuron can be "excited", how it can enter a "refractory" mode, and other consequences of the nonlinear interactions and time-scales in the model.

A. Basic HH model

Recall that the Hodgkin-Huxley model can be written as:

$$\begin{aligned} C\dot{V} &= I - \bar{g}_K n^4 (V - E_K) - \bar{g}_{Na} m^3 h (V - E_{Na}) - \bar{g}_l (V - E_l) \\ \dot{n} &= \alpha_n(V)(1-n) - \beta_n(V) n \\ \dot{m} &= \alpha_m(V)(1-m) - \beta_m(V) m \\ \dot{h} &= \alpha_h(V)(1-h) - \beta_h(V) h \end{aligned}$$

where the changes in the gating variables are modulated by 'on' and 'off' rates that are themselves voltage dependent (see equations above). The parameters are defined in Hodgkin and Huxley's original paper and already embedded in the codes.

This laboratory leverages a pre-developed HH simulator. The code in Python has four components

- `master_hh.py` – the master script for running the HH model
- `impulse_t.py` – a function that specifies customized current impulses
- `model_hh.py` – the dynamical system which calculates the change of each of the variables with time
- `plot_hh.py` – an output program which encapsulates the dynamics in a series of pre-formed panels, using the style and axis scales of Eugene Izhikevich

It is strongly encouraged to type in the model and master files, as doing so will help you think about the consequences of the particular relationships. However, the complete file set is already available for download on the laboratory home page. If you choose to download them, then you are *strongly encouraged* to slowly read them, i.e., over a 5-10 minute period.

Note that the formulation of the HH problem presents a challenge. The gating dynamics include on and off rates that include forms like $f(V)/g(V)$ where $g(V) = 0$ for some finite voltage V_s. This would seem to imply a singularity, i.e., divergence in the rate. But $f(V)$ also goes to 0 as $V \to V_s$, such that the ratio has a well defined limit. Numerical solvers do not necessarily know that. Hence, for some solvers, the simulations of the HH model crash or do not behave correctly, e.g. giving spurious or non-repeatable results. Hence, there are also two alternative files that you can/should use instead in the event of abnormal behavior. Read through and try to figure out what this alternative code set does to avoid this "singular" problem:

- `master_alt_hh.py` – the master script for running the HH model, while avoiding divide by 0 errors
- `model_alt_hh.py` – the dynamical system which calculates the change of each of the variables with time, while avoiding divide by 0 errors

The details of the solution to avoid divide by zero errors are described in the Appendices to this lab.

B. The key master and dynamics codes

Here is the master code that sets parameters and runs the model. You will note that a few pieces are missing.

```
# master_hh
# Script for HH models
#
# Simulates the HH model using parameters embedded in pars
# a stimulus current, and a time range over which the simulation should run

pars={}
```

```
# Parameters, including on/off functions
pars['gKbar'] = 36    # mS/cm^2
pars['gNabar'] = 120  # mS/cm^2
pars['gL'] = 0.3      # mS/cm^2
pars['EK'] = -12       # mV
pars['ENa'] = 120     # mV
pars['EL'] = 10.6     # mV
pars['C'] = 1         # muF/cm^2
pars['alphan'] = lambda V: 0.01*(10-V)/(np.exp(1-V/10)-1)
pars['betan'] = lambda V: 0.125*np.exp(-V/80)
pars['alpham'] = lambda V: 0.1*(25-V)/(np.exp(2.5-V/10)-1)
pars['betam'] = lambda V: 4*np.exp(-V/18)
pars['alphah'] = lambda V: 0.07*np.exp(-V/20)
pars['betah'] = lambda V: (np.exp(3-V/10)+1)**-1

# Initial conditions
pars['V0'] = 0
pars['n0'] = 0
pars['m0'] = 0
pars['h0'] = 0

# Run the model
t0 = 0
tf = 20
tstep = 0.02
t = np.arange(t0,tf+tstep,tstep)
y0 = np.array([pars['V0'], pars['n0'], pars['m0'], pars['h0']])
y = integrate.odeint(model_hh,y0,t,args=(pars,))

# Store the results
# FILL IN WHEREVER YOU SEE ...
dyn = {}
dyn['t'] = t
dyn['V'] = y[:,0]
dyn['n'] = y[:,1]
dyn['m'] = y[:,2]
dyn['h'] = y[:,3]
dyn['gK'] = pars['gKbar']*...
dyn['gNa'] = pars['gNabar']*...
dyn['gL'] = pars['gL']*...
dyn['IK'] = ...
dyn['INa'] = ...
dyn['IL'] =...
dyn['appliedI'] = impulse_t(dyn['t'])
```

Challenge Problem: Completing the HH setup.

Download the HH code package and then fill in the missing pieces of the `master` code whenever you see an ellipses (...).

Solutions to Challenge Problem: Completing the HH setup.

The code to insert should replicate the intent of the HH equations, including the nonlinear dependencies of gates as well as the currents as a function of the gating variables and voltages.

```
dyn['gK'] = pars['gKbar']*dyn['n']**4
dyn['gNa'] = pars['gNabar']*dyn['m']**3*dyn['h']
dyn['gL'] = pars['gL']*np.ones(len(dyn['t']))
dyn['IK'] = pars['gKbar'] * dyn['n']**4 * (dyn['V']-pars['EK'])
dyn['INa'] = pars['gNabar'] * dyn['m']**3 * dyn['h'] * (dyn['V']-pars['ENa'])
dyn['IL'] = pars['gL'] * (dyn['V']-pars['EL'])
```

Challenge Problem: Completing the HH equations.

Download the HH code package and then fill in the missing pieces of the `model` code whenever you see an ellipses (...).

```
# model_hh
def model_hh(y,t,pars):
    '''
    function dydt = model_hh(y,t,pars)
    This simulates the HH model using parameters embedded in pars
    a stimulus current in pars['Idrive']
    '''

    # Variables
    V = y[0]
    n = y[1]
    m = y[2]
    h = y[3]

    # Impulses
    I = impulse_t(t) # Specified in a function

    # Dynamics
    Vdot = (1/pars['C'])*(I-...) # FILL IN
    ndot = ...
    mdot = ...
    hdot = ...

    # Fill in
    dydt = np.array([Vdot, ndot, mdot, hdot])
    return dydt
```

Solutions to Challenge Problem: Completing the HH equations
The $\dot{V}$ equation as well as the dynamics of gating variables are taken directly from the HH equations and should read:

```
Vdot = (1/pars['C'])*(I-pars['gKbar']*n**4*(V-pars['EK'])-pars['gNabar']*m**3*h*(V-pars['ENa']) \
        -pars['gL']*(V-pars['EL']))
ndot = pars['alphan'](V)*(1-n)-pars['betan'](V)*n
mdot = pars['alpham'](V)*(1-m)-pars['betam'](V)*m
hdot = pars['alphah'](V)*(1-h)-pars['betah'](V)*h
```

It is important to note that in Python, the backslash \ represents a formal Python command; it means that the statement continues on the next line! Hence, although the code looks strange above, it will be accepted by the Python interpreter. However, there can be singularities given the voltage-dependent on/off rates of gates. The end of this lab includes technical details on how to interpolate across an apparent singularity in the numerical system of equations. This is implemented in the code available for download.

The script and function used to numerically integrate the Hodgkin Huxley model also requires the application of an applied current, $I(t)$. Previous labs demonstrated how to include time-dependent parameters in the numerical integration of ODEs. Here, you should do the same, and consider saving different versions of applied current functions for use and reuse.

Challenge Problem: Time-dependent Impulses

Develop an impulse function in which the applied current is

$$\begin{aligned} I(t) &= 2 && 2 \leq t \leq 2.5 \\ I(t) &= 25 && 10 \leq t \leq 10.5 \\ I(t) &= 0 && \text{otherwise} \end{aligned}$$

all in units of μA/cm^2. In doing so, modify the following impulse code, that enables time-specific additions of current.

```
# impulse_t
def impulse_t(t):
    '''
    function I = impulse_t(t)

    specifies the applied time-varying current
    works if t is a single value or many values
    '''

    if isinstance(t,float):
        if t>2 and t<2.5:
            I=...
        elif t>10 and t<10.5:
            I=...
        else:
            I=...
    else:
        I=np.zeros(len(t))
        for i in range(len(t)):
            if t[i]>2 and t[i]<2.5:
                I[i] = ...
            elif t[i]>10 and t[i]<10.5:
                I[i] = ...
            else:
                I[i]=...
    return I
```

Solutions to Challenge Problem: Time-dependent Impulses
Here, the code serves double duty. If there is only a single value of `t`, as in the numerical integration, then only a single applied current value will be returned. However, if many values are input, then the code will return the entire function, this is useful for plotting and storing the applied current along with the output.

```
# impulse_t
def impulse_t(t):
    '''
    function I = impulse_t(t)

    specifies the applied time-varying current
    works if t is a single value or many values
    '''

    if isinstance(t,float):
        if t>2 and t<2.5:
            I=2
        elif t>10 and t<10.5:
            I=25
        else:
            I=0
    else:
        I=np.zeros(len(t))
        for i in range(len(t)):
            if t[i]>2 and t[i]<2.5:
                I[i] = 2
            elif t[i]>10 and t[i]<10.5:
                I[i] = 25
            else:
                I[i]=0
    return I
```

C. A digression on plotting multiple axes

The additional codes include plotting capabilities. These plotting codes are essential, but they are also stylized given that they were developed for a particular, aesthetic output, but not necessarily for straightforward reading. There are good ideas here on how to stack axes that should be read and reviewed with care. In addition, the technical appendices to this lab include additional information on aesthetic choices.

Challenge Problem: Customizing Axes in a Single Figure

Plot the voltage-sensitive gating functions as stacked plots, with a common shared x-axis, each on its own axes. In Python, you can create both a figure handle and an array of axes with the `plt.subplot` command. When you give this command, you have the option of resizing the figure to make room for multiple axes as in

```
fig, axes = plt.subplots(numRows,numCols,figsize = (figWidth,figHeight))
```

and you can access the individual axes from the array.

```
ax0=axes[0]
```

You can also eliminate tick labels (useful when axes share the same x values), as follows:

```
ax0.set_xticklabels([])
```

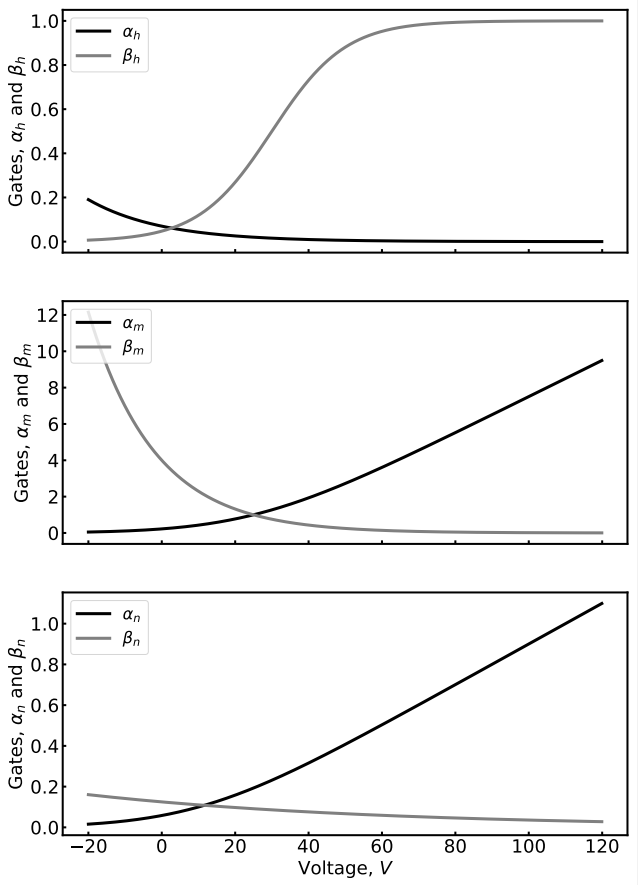
This figure is large, and may be hard to view in the console. If you would like the figure to appear in a separate window, give the command

```
%matplotlib qt
```

in the console. To switch back to viewing the figures in the console, give the command

```
%matplotlib inline
```

If your code is working, it should look like the image below.



Solutions to Challenge Problem (part 1 of 2): Customizing Axes in a Single Figure.
The following solution combines two features. First, one must deal with the singularity in gate variables to produce a bona fide output without NAN results (NAN = not a number). Second, axes can be stacked in multiple different ways, however a useful figure code is one that looks the same each time it is run. To do so it is often prudent to specify both the size and layout of the figure window as well as the size and position of each axis. The solution is presented in two parts: (i) the calculation elements (in this box); (ii) axes customization (in the next box).

```
import numpy as np
import matplotlib.pyplot as plt

#main data goes here
V = np.arange(-20,120,0.1)
pars={}
pars['alphan'] = lambda V: 0.01*(10-V)/(np.exp(1-V/10)-1)
pars['betan'] = lambda V: 0.125*np.exp(-V/80)
pars['alpham'] = lambda V: 0.1*(25-V)/(np.exp(2.5-V/10)-1)
pars['betam'] = lambda V: 4*np.exp(-V/18)
pars['alphah'] = lambda V: 0.07*np.exp(-V/20)
pars['betah'] = lambda V: (np.exp(3-V/10)+1)**-1

#set the gates
#n
alphan_eval = np.zeros(np.shape(V))
tmpi = np.argwhere(V!=10)
alphan_eval[tmpi] = pars['alphan'](V[tmpi])
tmpi = np.argwhere(V==10)
alphan_eval[tmpi]=0.1

#m
alpham_eval = np.zeros(np.shape(V))
tmpi = np.argwhere(V!=25)
alpham_eval[tmpi] = pars['alpham'](V[tmpi])
tmpi = np.argwhere(V==25)
alphan_eval[tmpi]=1

#h
alphah_eval = pars['alphah'](V)
```

Solutions to Challenge Problem (part 2 of 2): Customizing Axes in a Single Figure
The layouts will vary by taste. The following code stacks each set of gating variables and compares the α and β values associated with each gate, corresponding to on and off rates, respectively.

```
fig, axes = plt.subplots(3,1,figsize = (10,15))
# Plot - n gates
axn = axes[2]
axn.plot(V,alphan_eval,color='k',linewidth=3)
axn.plot(V,pars['betan'](V),color=[0.5,0.5,0.5],linewidth=3)
axn.set_ylabel(r'Gates, $\alpha_n$ and $\beta_n$',fontsize=18)
axn.set_xlabel(r'Voltage, $V$',fontsize=18)
axn.legend([r'$\alpha_n$',r'$\beta_n$'],fontsize=16,loc='upper left')
axn.set_yticks(np.arange(0,12,2)*0.1)
plt.setp(axn.spines.values(),linewidth=2)
axn.tick_params(labelsize = 18,width=2,direction='in')

#plot - m gates
axm = axes[1]
axm.plot(V,alpham_eval,color='k',linewidth=3)
axm.plot(V,pars['betam'](V),color=[0.5,0.5,0.5],linewidth=3)
axm.set_ylabel(r'Gates, $\alpha_m$ and $\beta_m$',fontsize=18)
axm.legend([r'$\alpha_m$',r'$\beta_m$'],fontsize=16,loc='upper left')
axm.set_yticks(np.arange(0,14,2))
axm.set_xticklabels([])
plt.setp(axm.spines.values(),linewidth=2)
axm.tick_params(labelsize = 18,width=2,direction='in')

axh = axes[0]
axh.plot(V,alphah_eval,color='k',linewidth=3)
axh.plot(V,pars['betah'](V),color=[0.5,0.5,0.5],linewidth=3)
axh.set_ylabel(r'Gates, $\alpha_h$ and $\beta_h$',fontsize=18)
axh.legend([r'$\alpha_h$',r'$\beta_h$'],fontsize=16,loc='upper left')
axh.set_yticks(np.arange(0,12,2)/10)
axh.set_xticklabels([])
plt.setp(axh.spines.values(),linewidth=2)
axh.tick_params(labelsize = 18,width=2,direction='in')
```

III. FIRING WITHOUT A CURRENT

Once you have the code entered or downloaded, you should be able to run the model, as follows in Python by compiling `master_hh.py`. The resulting plot takes up a lot of space, making it difficult to view in the console. If all goes well, you should get the result on the figure below (see left panels). As is apparent the neuron seems to begin to fire despite the fact that no current is applied. In addition, the neuron seems refractory with respect to the larger applied current at 10 ms. In contrast, the same model, using the same parameters yields the "classic" neuronal excitation dynamics as seen in the result on the right panel of the figure below. The plotting code does include the detailed annotations as did Izhikevich, however you should already notice a number of differences.

Thought problem for reflection

Identify the difference between the initial run and that shown in the book. What is the biological meaning of the difference and why does the neuron "fire" before a signal arrives? Can you identify a "fix" to the Python model that would make it behave "correctly"?
Spoiler alert next page...

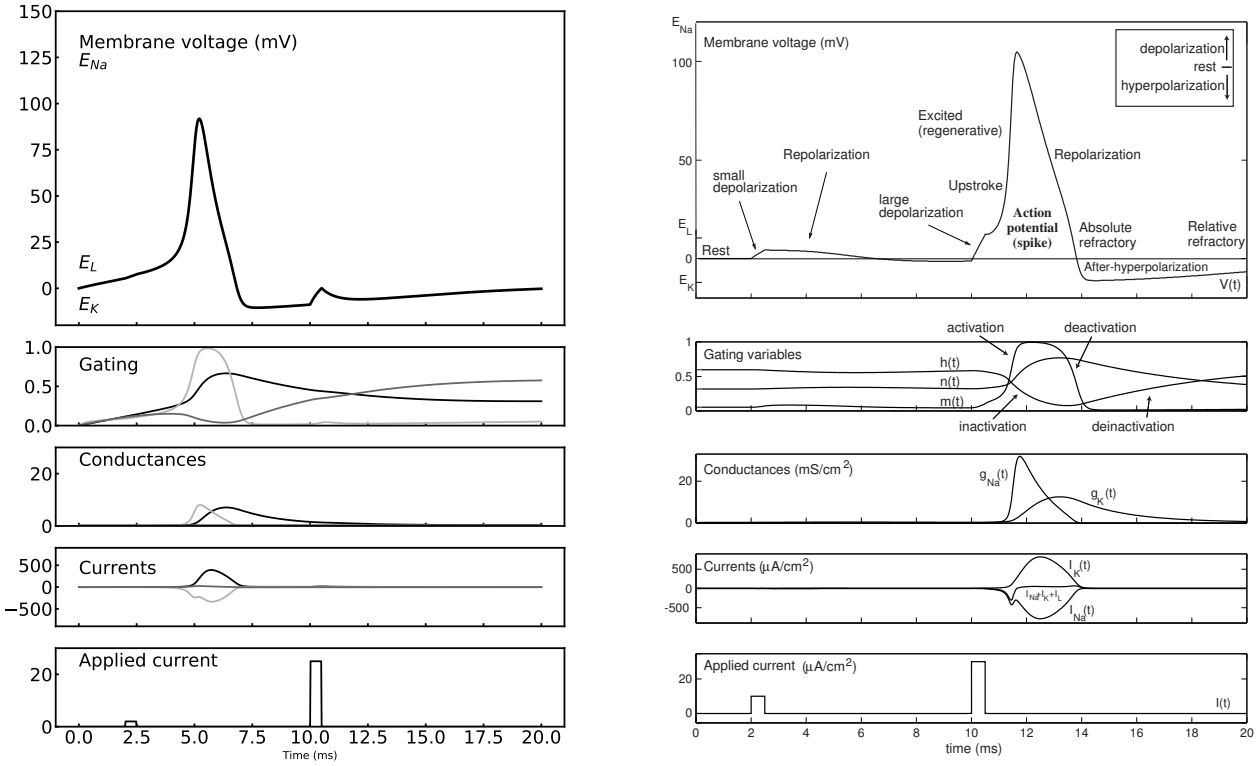


FIG. 2: Contrasting results from simulating the Hodgkin-Huxley model. (Left) Results from the pre-supplied Python code; (Right) Results from Izhikevich's book on Dynamical Neuroscience (Figure 2.15).

The model developed here as shown in the left panel and the HH model as implemented in the right hand panel are exactly the same. The applied currents are, more or less, the same. The initial voltage is the same. However, in the model that was initially supplied, the initial gating states are all set to 0, i.e., they are all inactive. This is not the equilibrium condition of the model. Note that at equilibrium

$$\begin{aligned} n^* &= \alpha_n(V^*)/\left(\alpha_n(V^*)+\beta_n(V^*)\right) \\ m^* &= \alpha_m(V^*)/\left(\alpha_m(V^*)+\beta_m(V^*)\right) \\ h^* &= \alpha_h(V^*)/\left(\alpha_h(V^*)+\beta_h(V^*)\right) \end{aligned}$$

At these particular values, the system will be at rest. If not, then the gates will begin to open, conductances will increase, the membrane depolarizes and then repolarizes and the system will appear to fire even if no stimulus or a weak stimulus is applied. If one uses the equilibrium values as the initial conditions, then one recovers the following (see Figure 3). As is evident, the neuron does not fire given the first applied current but does so in response to the second, larger applied current at 10ms.

Challenge Problem: Filtering and Excitation

Identify the equilibrium voltage and gating variables for the HH equations. Then, using the same impulse function as before see if the system can filter out small input currents and react in an excitatory fashion to large inputs, as demonstrated in Figure 3.

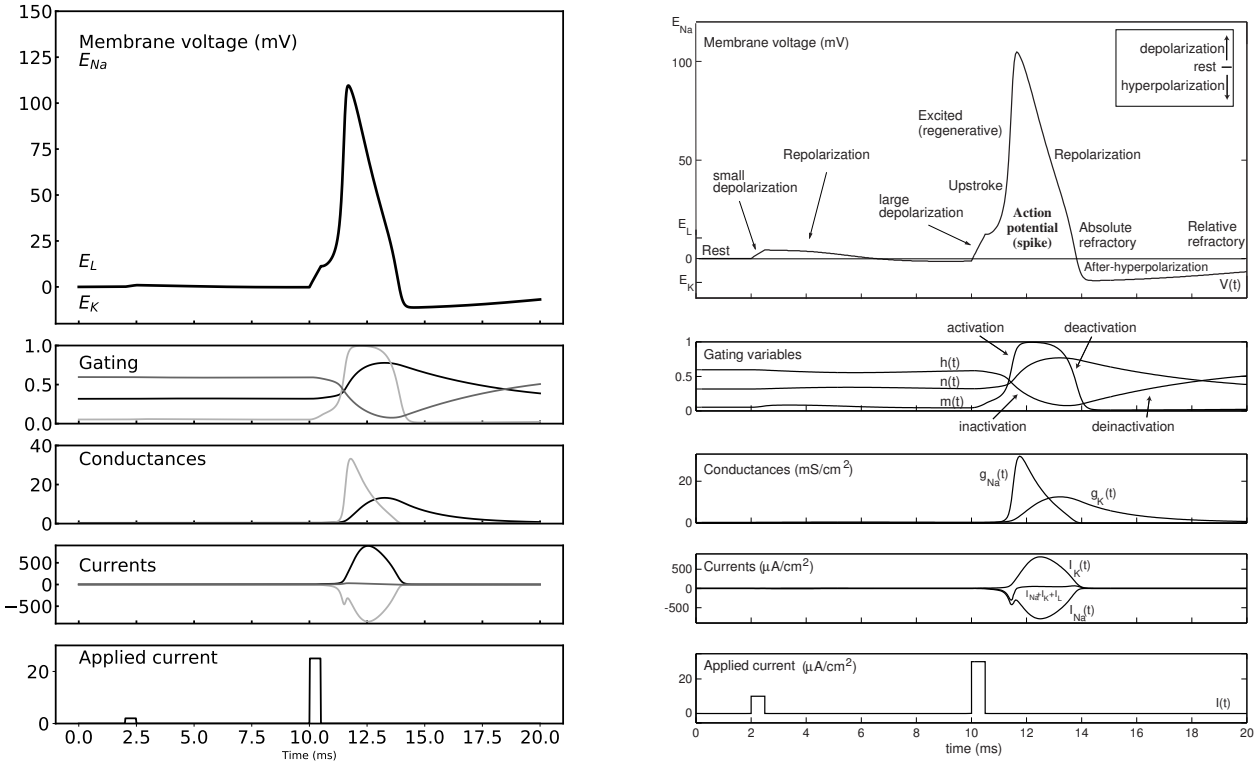


FIG. 3: Agreeing results from simulating the Hodgkin-Huxley model. (Left) Results from the pre-supplied Python code; (Right) Results from Izhikevich's book on Dynamical Neuroscience (Figure 2.15).

Solutions to Challenge Problem: Filtering and Excitation.
There are different ways to identify the equilibrium. One way is to simulate the model until it relaxes back to steady state and then utilize these equilibria voltage and gating states as your initial conditions. Another way is to solve for the equilibrium concentrations directly, the key here is to recognize that at equilibrium, then $V^* = 0$, i.e., there is a zero resting potential. Hence, for the n variable at steady state

$$\alpha_n(V^*)(1-n) = \beta_n(V^*)n \tag{11}$$

which leads to the solution $n^* = \frac{\alpha_n(0)}{\alpha_n(0)+\beta_n(0)}$. Similar logic can be implemented for all the variables as follows:

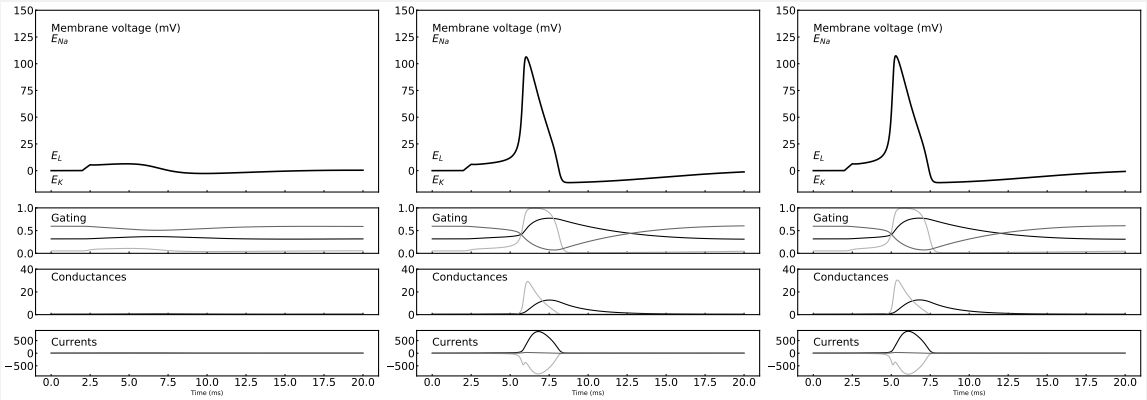
```
pars['V0']=0
pars['n0']=pars['alphan']*pars['V0']/(pars['alphan']*pars['V0']+pars['betan']*pars['V0'])
pars['m0']=pars['alpham']*pars['V0']/(pars['alpham']*pars['V0']+pars['betam']*pars['V0'])
pars['h0']=pars['alphah']*pars['V0']/(pars['alphah']*pars['V0']+pars['alphah']*pars['V0'])
```

IV. NEURON DYNAMICS: THRESHOLDS IN MAGNITUDE AND TIME

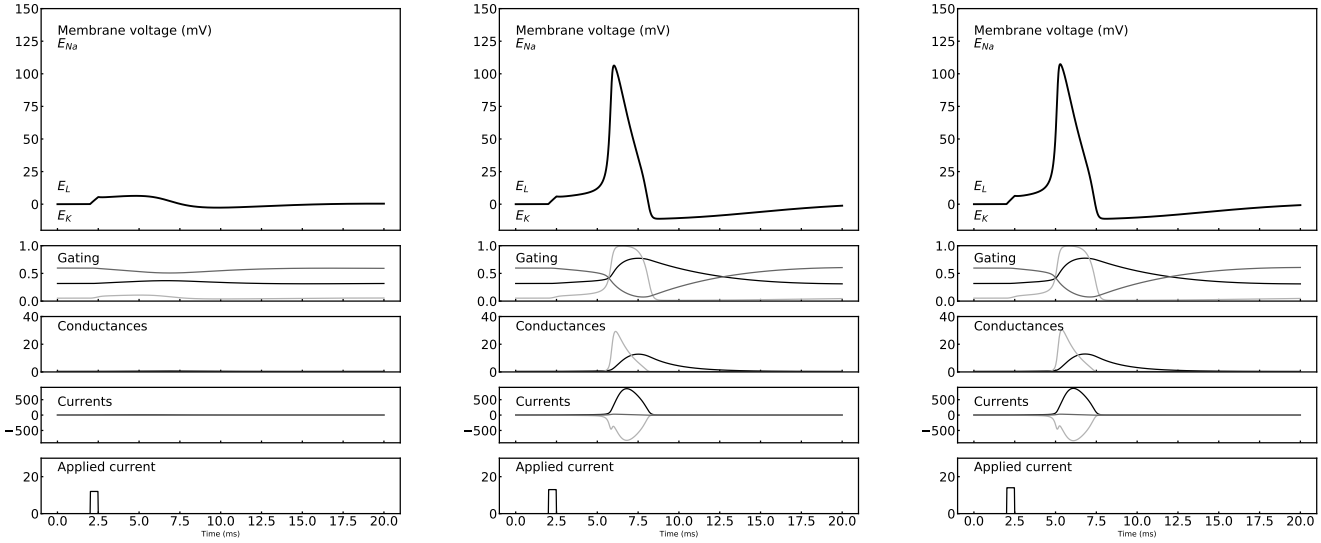
Now that you have a functioning HH model, it is time to explore. The focus of these last two challenge problems is to modify the input $I(t)$ in different ways to identify how a neuron can exhibit threshold behavior given variation in the magnitude of an incoming pulse as well as the spacing between pulses. However, please do not feel constrained by only these two challenge problems. There is no way to fully cover the scope of neuronal dynamics in this laboratory and self-guided exploration of the model is essential.

Challenge Problem: Thresholds for Firing

Modify the code to identify the minimum current required for firing. Such current can depend on the duration of the pulse. Here, identify the minimum applied current over a period of 0.5 msec required to 'fire' the neuron: so that the values within 1 μ A/cm^2 do not fire? If so, you should find a result as in Figure 3 – where the bottom $I(t)$ panel has been removed.



Solutions to Challenge Problem: Thresholds for Firing. Here, systematically varying $I(t)$ in increments of 1 identify a switch in behavior from $I = 12$ (left) to $I = 13$ (middle). The firing continues beyond this point, e.g., when $I = 14$ (right).



Next, modify the impulses to explore the effects of the timing interval between applied current and the resulting firing of neurons. An example is provided in Figure 4. In it, you will see that there is a super-critical applied current that drives an initial action potential. In the two scenarios - the second applied current pulse has markedly different effects on the next action potential.

Challenge Problem: Refractory Periods (bonus)

What is the minimum interval required to induce a second action potential spike in the neuron after the first applied current? Note that the answer to this problem is not provided here, it is ideally suited for in-depth exploration.

There are many other ways to explore neuronal dynamics and excitation. Here are just a few possibilities:

- The neuronal dynamics exhibit many features, including "hyperexcitability". Can you identify a period after the initial pulse where even *less* current is required to generate an action potential spike?

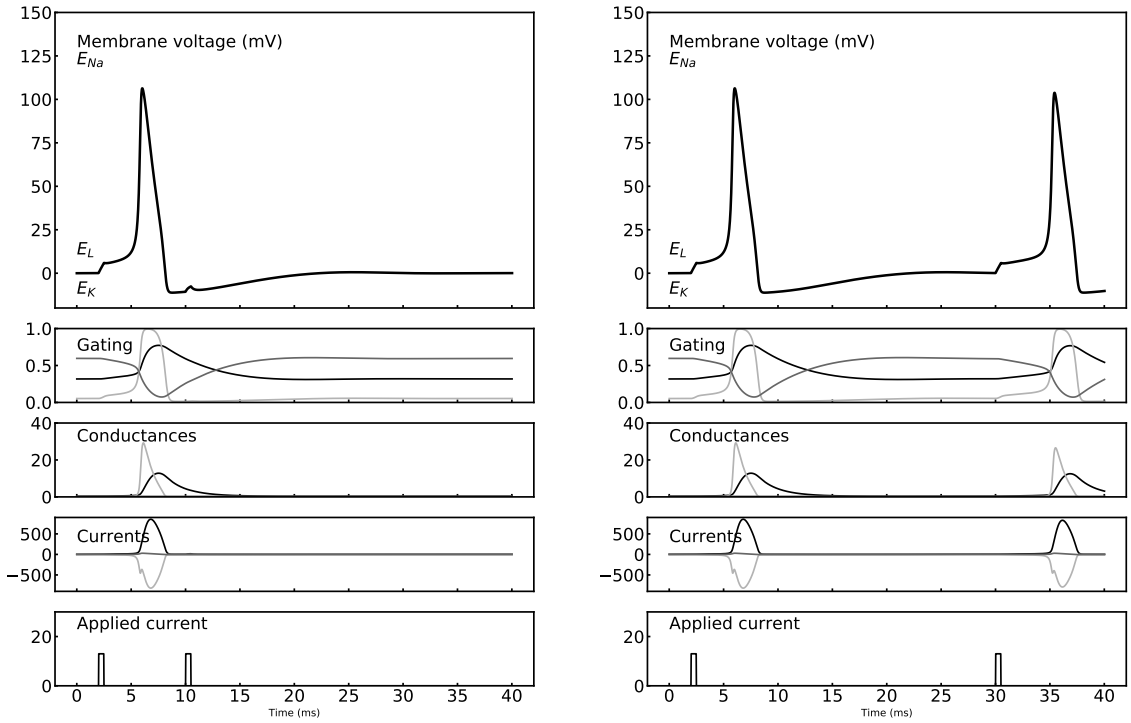


FIG. 4: Firing dynamics at sub-critical and super-critical levels. In this case a current is applied form 2-2.5 ms with the magnitude differing only by 1.

- Can you characterize the time-scales of relaxation of the activation and inactivation gates? How do they differ as a function of voltage?
- How do the period and peak height of the action potential vary with the intensity of the applied current? Is the relationship linear or nonlinear?
- Explore the effect of additional impulses of different duration and magnitudes while keeping the product of the current times the time the same. Are all "pulses" equal? Can a long weak pulse generate a signal? Can a long but very strong pulse generate a signal?

V. TECHNICAL APPENDIX

A. Avoiding apparent 'singularities' in the HH integration

The HH model includes voltage-sensitive gates, including two that pose issues for numerical integration:

$$\alpha_n(V) = 0.01\frac{10-V}{e^{(10-V)/10}-1} \tag{12}$$

and

$$\alpha_m(V) = 0.1\frac{25-V}{e^{(25-V)/10}-1}. \tag{13}$$

The issue for both is that the denominators go to 0, e.g., $V = 10$ in the case $\alpha_n(V)$ and $V = 25$ in the case of $\alpha_m(V)$. Yet, in neither case does the actual value of the gate-specific on rate diverge. Instead, in both cases the numerator and the denominator go to 0 as $V \to V_s$ (the apparent singularity point). In order to understand the behavior, one needs to understand the limit of these functions, as $V \to V_s$. For example, consider, $\alpha_n(V)$. In that case, then we can use l'Hopital's rule, i.e., if $q(x) = f(x)/g(x)$ then

$$\lim_{x\to x_0} q(x) = \frac{f'(x)|_{x=x_0}}{g'(x)|_{x=x_0}} \tag{14}$$

where $f'(x)$ and $g'(x)$ denote derivatives with respect to x. In which case,

$$\lim_{V\to 10} = 0.01\frac{-1}{-1/10 \times e^{(10-V)/10}|_{V=10}} \tag{15}$$

which means $\alpha_n(10) = 0.1$. Similarly, then $\alpha_m(25) = 1$. The figures below denote the two nonlinear gating functions, highlighting the replacement values.

It would seem that one could run a numerical integrator, but always utilize a replacement if the integration ever passed precisely on the value of the apparent singularity. Not all numerical integrators work this way, i.e., replacing just one point rather than a continuous set of points poses a problem for schemes. Instead, one way is to use a piecewise smooth version of the functions. The following code replaces the original version shown in this lab, i.e., it is a bit less intuitive, but it is also more reliable.

```
# Define the expected voltage limits
pars['Vmin'] = -100
pars['Vmax'] = 200
pars['dV'] = 0.1
V = np.arange(pars['Vmin'],pars['Vmax']+pars['dV'],pars['dV'])
pars['V'] = V

# Define the original functions, with piecewise smooth functions
pars['alphan'] = 0.01*(10-V)/(np.exp(1-V/10)-1)
pars['alpham'] = 0.1*(25-V)/(np.exp(2.5-V/10)-1)
# Anonymous functions
pars['alphah'] = lambda V: 0.07*np.exp(-V/20)
pars['betan'] = lambda V: 0.125*np.exp(-V/80)
pars['betam'] = lambda V: 4*np.exp(-V/18)
pars['betah'] = lambda V: (np.exp(3-V/10)+1)**-1

# Assign the l'Hopital limit to those values where
# Python thinks the function is 'Not a Number' (NAN)
tmpi = np.where(np.isnan(pars['alphan']))[0]
pars['alphan'][tmpi]=0.1
tmpi = np.where(np.isnan(pars['alpham']))[0]
pars['alpham'][tmpi]=1
```

With the script in hand, the function invoked by `integrate.odeint` must also change. the derivatives must now utilize the interpolated values of `pars['alphan']` and `pars['alpham']` The following is in the core of the function `model_hh` in which the following steps are taken. First, the index of the interpolated array of α_n and α_m is found. Next, the value of α_n and α_m is found by assuming that these rates can be lineally interpolated on a tight grid. Finally, the values are utilized for the exact same set of HH equations as before.

```
# model_hh
def model_hh(y,t,pars):
    '''
    function dydt = model_hh(y,t,pars)
    This simulates the HH model using parameters embedded in pars
    a stimulus current in pars['Idrive']
    '''
    ...

    # Fixes the issue with the singularity in the alphan/alpham
    # The intuition here is to find the right index for
    # the current V value, and then interpolate to make it piecewise
    # smooth. This method also avoids using ''where'' and ''if'' commands
    # to ensure reasonable speed.
    vmin_ind = 1+(V-pars['Vmin'])/pars['dV']
    tmpi = int(np.floor(vmin_ind))
    dev_ind = vmin_ind - tmpi
    alphan = pars['alphan'][tmpi] + (pars['alphan'][tmpi+1]-pars['alphan'][tmpi])*dev_ind
    alpham = pars['alpham'][tmpi] + (pars['alpham'][tmpi+1]-pars['alpham'][tmpi])*dev_ind

    # Dynamics
    Vdot = (1/pars['C'])*(I-pars['gKbar']*n**4*(V-pars['EK'])-pars['gNabar']*m**3*h*(V-pars['ENa'])-pars['gL']*(V-pars['EL']))
    ndot = alphan*(1-n)-pars['betan'](V)*n
    mdot = alpham*(1-m)-pars['betam'](V)*m
    hdot = pars['alphah'](V)*(1-h)-pars['betah'](V)*h
```

```
    # Fill in
    dydt = np.array([Vdot, ndot, mdot, hdot])
    return dydt
```

B. Plotting – it takes time

Plotting – in whatever language – takes time and effort. The beginning coder is encouraged to hew close to a few basic guidelines, and perhaps read a bit of Edward Tufte for inspiration. We recommend a few simple guidelines:

- Every figure with data has its own scripted figure file, i.e., if the figure was worth making once, it may be worth making twice, and you should be able to do the exact same thing again!
- The ratio of data/ink should be high. That is to say: white space should be respected. As but one example, if you have a 'bar' chart, consider replacing the tops of ink-filled bars with single points. Same data, less ink. Apply principle repeatedly until satisfied.
- Ranges and axes should be used to help guide the reader, i.e., judicious use of axes ranges and scaling (log or linear) can help guide a reader to improved interpretation.
- Use color. Papers are read online (and for those reading this book; we concede that the constraints of the publication process into a physical book mean that the bulk of the figures in these laboratories are in black and white, feel free not to be constrained by such a limitation in your own work).
- Legends, labels, and units. Use them.

With this pre-amble in mind, the following command generates a plot and returns a handle based on the dynamics embedded in `dyns` and the parameters in `pars` resulting from a HH simulation.

```
def plot_hh(dyn,pars,savePDF=False,figName=''):
    '''
    function ploth = plot_hh(dyn, pars)

    Takes a HH output structure in dyn and plots dynamics of Voltage, gating variables, conductance
    currents and applied current
    '''

    fig, axes = plt.subplots(5,1,figsize = (9,12),gridspec_kw = {'height_ratios':[4,1,1,1,1]})



    # main data goes here

    # Applied current on bottomA
    ax4 = axes[4]
    ax4.plot(dyn['t'],dyn['appliedI'],color='k', linestyle='-',linewidth=2)
    ax4.set_xlabel('Time (ms)',fontsize=12)
    ax4.text(0,23,'Applied current',fontsize=20)
    ax4.set_ylim([0,30])
    plt.setp(ax4.spines.values(),linewidth=2)
    ax4.tick_params(labelsize = 20,width=2,direction='in')

    # Next up is Currents
    ax3 = axes[3]
    ax3.plot(dyn['t'],dyn['IK'],color='k')
    ax3.plot(dyn['t'],dyn['INa'],color='r')
    ax3.plot(dyn['t'],dyn['IL'],color='b')
    ax3.text(0,300,'Currents',fontsize=20)
    ax3.set_ylim([-900,900])
    plt.setp(ax3.spines.values(),linewidth=2)
    ax3.tick_params(labelsize = 20,width=2,direction='in')
    ax3.set_xticklabels([])

    # Conductance
    ax2 = axes[2]
    ax2.plot(dyn['t'],dyn['gK'],color='k')
    ax2.plot(dyn['t'],dyn['gNa'],color='r')
```

```
ax2.plot(dyn['t'],dyn['gL'],color='b')
ax2.text(0,23,'Conductances',fontsize=20)
ax2.set_ylim([0,30])
plt.setp(ax2.spines.values(),linewidth=2)
ax2.tick_params(labelsize = 20,width=2,direction='in')
ax2.set_xticklabels([])

#Gating
ax1 = axes[1]
ax1.plot(dyn['t'],dyn['n'],color='k')
ax1.plot(dyn['t'],dyn['m'],color='r')
ax1.plot(dyn['t'],dyn['h'],color='b')
ax1.text(0,0.7,'Gating',fontsize=20)
ax1.set_ylim([0,1])
plt.setp(ax1.spines.values(),linewidth=2)
ax1.tick_params(labelsize = 20,width=2,direction='in')
ax1.set_xticklabels([])

# Voltage
ax0 = axes[0]
ax0.plot(dyn['t'],dyn['V'],color='k',linewidth=3)
ax0.text(0,130,'Membrane voltage (mV)',fontsize=20)
ax0.set_ylim([-20,150])
ax0.text(0,pars['EK'],r'$E_K$',fontsize=20)
ax0.text(0,pars['ENa'],r'$E_{Na}$',fontsize=20)
ax0.text(0,pars['EL'],r'$E_L$',fontsize=20)
plt.setp(ax0.spines.values(),linewidth=2)
ax0.tick_params(labelsize = 20,width=2,direction='in')
ax0.set_xticklabels([])


plt.tight_layout()

return fig
```